CS24301 — THEORY (3.0 CR) FULL COURSE MASTER BIT MESRA NEP SCHEME

Compiler Design (CS24301) — Full Course Master Book

Complete End-to-End B.Tech CSE 5th Semester Study Book & PYQ Bank

EXECUTIVE MASTER STUDY GUIDE & PYQ BANK

Compiler Design (CS24301)

Birla Institute of Technology, Mesra | B.Tech CSE 5th Semester (NEP 2024-25 Scheme)

📚 Complete Course Structure & Syllabus Matrix

Module	Core Syllabus Scope	Key Algorithms & Formulations
Module I	Lexical Analysis & Automata	6 Phases, Two-Buffer Scheme, Thompson RE to NFA, Subset Construction, Hopcroft DFA Minimization
Module II	Syntax Analysis & Parsers	CFG, Left Recursion/Factoring, FIRST & FOLLOW, LL(1) Table, Shift-Reduce, LR(0), SLR(1), CLR(1), LALR(1)
Module III	Semantic Analysis & Types	Syntax-Directed Definitions (SDD), Synthesized/Inherited, S/L-Attributed, Type Checking, Symbol Tables
Module IV	Intermediate Code & Runtime	Three-Address Code (Quadruples/Triples), Multi-D Array Addressing, Backpatching, Activation Records
Module V	Optimization & Code Gen	Basic Blocks, CFG, DAG, Loop Invariant Code Motion, Reaching Definitions, Register Coloring

🎯 Exam Preparation & High-Yield Guide

This publication-grade master book consolidates all 5 modules with formal mathematical proofs, KaTeX-rendered formulas, step-by-step worked traces on classic datasets, and comprehensive model answers to BIT Mesra end-semester examination questions.

Module I: Lexical Analysis & Automata Theory — Complete Syllabus Topics

1. Language Processors (Compiler, Interpreter, Assembler, Preprocessor, Linker, Loader)
2. 6 Phases of Compiler Architecture with Full Running Translation Traces
3. Compiler Front-End vs. Back-End & Pass Structures (1-Pass vs. Multi-Pass)
4. Lexical Analysis Role, Tokens, Patterns, Lexemes & Token Attributes
5. Input Buffering Mechanics (Two-Buffer Scheme & Sentinel Verification)
6. Regular Expressions & Regular Definitions for Programming Language Tokens
7. Thompson's Construction Algorithm (RE to ϵ-NFA with Structural Rules)
8. Subset Construction Algorithm (NFA to DFA Transition Table Derivations)
9. Direct DFA Construction from RE (Nullable, Firstpos, Lastpos, Followpos)
10. Hopcroft's DFA State Minimization Algorithm (Equivalence Partitioning Trace)
11. Lex & Flex Lexical Analyzer Generator Architecture & Ambiguity Rules
12. Comprehensive Solved BIT Mesra & GATE Question Bank (8 Solved Problems)

Topic 1: Language Processing Systems & Execution Pipeline

A **Compiler** is a sophisticated system software program that accepts a source program written in a high-level, human-readable programming language (such as C, C++, Rust, or Java) and translates it into an equivalent semantic representation in a low-level target language (machine code or assembly), while detecting and reporting all syntax and static semantic violations.

Figure 1.1: Complete End-to-End Language Processing Pipeline & System Tools

Detailed Analysis of System Components (The Cousins of the Compiler):

Tool	Input → Output	Primary Technical Responsibilities & Runtime Execution
1. Preprocessor	Raw High-Level Code → Pure High-Level Code	• **Macro Expansion:** Replaces macro identifiers with their replacement text (e.g., `#define PI 3.14159`). • **File Inclusion:** Injects verbatim contents of standard or user header files (e.g., `#include `). • **Conditional Compilation:** Evaluates compile-time conditionals (e.g., `#ifdef DEBUG ... #endif`) to strip unwanted source lines. • **Comment Stripping:** Removes all single-line `//` and block `/* ... */` comments from the source stream.
2. Compiler	Pure Source Code → Target Assembly Code	Performs full syntactic, semantic, and intermediate translation passes. Emits symbolic target assembly mnemonics (e.g., x86-64, ARM, RISC-V) rather than raw machine opcodes, allowing the assembler to handle platform-specific bit encodings.
3. Assembler	Assembly Mnemonics → Relocatable Object Code	Translates assembly instructions into raw binary machine opcodes. Constructs the initial **Relocatable Object File** (`.o` or `.obj`) containing machine instructions, initialized data tables, and an internal **Relocation Table** for external labels.
4. Linker	Relocatable Object Files + Static Libraries → Absolute Executable	• **Symbol Resolution:** Matches external function and variable references across separately compiled compilation units (e.g., linking `printf` calls to `libc.a`). • **Relocation:** Merges separate data and text sections into unified virtual address spaces, recalculating all relative memory offsets.
5. Loader	Absolute Executable on Disk → Running Process in RAM	Allocates primary memory space (Stack, Heap, Data, Text), loads machine instructions into RAM, sets up dynamic library linkages (`.so` / `.dll`), initializes hardware CPU registers (Instruction Pointer `RIP`, Stack Pointer `RSP`), and triggers execution.

Topic 2: The 6 Phases of Compiler Architecture

A modern compiler operates as a strict sequence of analytical (Front-End) and synthetic (Back-End) phases. Every phase transforms the program representation while maintaining bidirectional communication with the central **Symbol Table** and **Error Handler**.

🏛️ Complete End-to-End Tracing: Statement `position = initial + rate * 60`

To understand the continuous transformations, follow how the statement `position = initial + rate * 60` is processed across all 6 phases:

1. **Phase 1: Lexical Analysis (Scanning):**
   Reads raw character stream from left to right, partitions characters into valid lexemes, and emits a linear token sequence:

$$\langle \mathbf{id}, 1 \rangle \ \langle = \rangle \ \langle \mathbf{id}, 2 \rangle \ \langle + \rangle \ \langle \mathbf{id}, 3 \rangle \ \langle * \rangle \ \langle \mathbf{num}, 60 \rangle$$

   Symbol Table Entries: `1: position (float)`, `2: initial (float)`, `3: rate (float)`.

2. **Phase 2: Syntax Analysis (Parsing):**
   Verifies grammatical structure using Context-Free Grammars (CFG) and constructs a hierarchical **Abstract Syntax Tree (AST)** that inherently enforces operator precedence ($*$ binds tighter than $+$):

```
        =
       / \
  id(1)   +
         / \
    id(2)   *
           / \
      id(3)  60
```

3. **Phase 3: Semantic Analysis (Type Checking & Coercion):**
   Checks language semantic constraints (variable declarations, scope, operand type compatibility). Since `rate` is a `float` and `60` is an `integer`, it inserts an implicit type conversion operator node $\mathrm{inttofloat}$:

```
        =
       / \
  id(1)   +
         / \
    id(2)   *
           / \
      id(3)  inttofloat
                 |
                60
```

4. **Phase 4: Intermediate Code Generation (ICG):**
   Translates the decorated AST into machine-independent, linear **Three-Address Code (TAC)** where each instruction contains at most one operator:

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

5. **Phase 5: Machine-Independent Code Optimization:**
   Analyzes the TAC to eliminate redundant computations, evaluate constants at compile-time (Constant Folding), and reduce temporary register pressure:

```
t1 = id3 * 60.0        ; Constant folded inttofloat(60) → 60.0 at compile-time
id1 = id2 + t1         ; Eliminated unnecessary temporary t3
```

6. **Phase 6: Code Generation (Target Machine Assembly):**
   Maps intermediate variables to hardware CPU registers (`R1`, `R2`) and emits optimized target assembly instructions:

```
LDF    R2, id3          ; Load floating-point rate into register R2
MULF   R2, #60.0        ; Multiply R2 by constant literal 60.0
LDF    R1, id2          ; Load floating-point initial into register R1
ADDF   R1, R2           ; Add R2 into R1 (R1 = initial + rate * 60.0)
STF    id1, R1          ; Store result from R1 into memory location position
```

Topic 3: Compiler Passes & Architecture Organization

Front-End vs. Back-End Separation ($N \times M$ Retargetability Principle)

- **Front-End (Analysis):** Depends strictly on the source programming language (Lexical, Syntax, Semantic, and ICG). It produces a clean, machine-independent Intermediate Representation (IR).
- **Back-End (Synthesis):** Depends strictly on the target hardware architecture (Optimization, Register Allocation, Instruction Scheduling, Machine Code Generation).
- **Engineering Significance:** To build compilers for N high-level languages targeting M distinct CPU architectures, modular separation requires only N front-ends and M back-ends ($N + M$ modules), rather than $N \times M$ separate monolithic compilers!

Single-Pass vs. Multi-Pass Compilers:

Single-Pass Compiler: Reads source code once and generates target code directly without creating intermediate files. Highly memory efficient (used in early Pascal compilers), but cannot perform global optimizations or forward-referencing declarations without backpatching.
Multi-Pass Compiler: Traverses intermediate representations multiple times. Enables deep interprocedural optimizations, global register coloring, and dead code elimination at the cost of higher compilation latency and memory consumption.

Topic 4 & 5: Lexical Analysis Role, Token Definitions & Input Buffering

1. Essential Lexical Terminology:

Token: An abstract terminal category recognized by the compiler front-end (e.g., `KEYWORD`, `IDENTIFIER`, `NUMBER`, `RELOP`, `ASSIGN`).
Pattern: The formal grammatical specification (usually a Regular Expression) that character sequences must satisfy to match a token (e.g., `[a-zA-Z_][a-zA-Z0-9_]*`).
Lexeme: The concrete sequence of source characters that matches a token's pattern (e.g., `total_sum`, `3.14159`, `==`, `while`).
Token Attribute: Additional metadata stored alongside the token in the Symbol Table (e.g., type, memory offset, line number, scope depth).

2. Input Buffering Architecture (Two-Buffer Scheme with Sentinels):

Reading source files character-by-character via direct OS system calls incurs massive I/O overhead. A **Two-Buffer Scheme** allocates two contiguous N-byte blocks (typically $N = 4096$ bytes, matching the physical disk block size):

Figure 1.2: Two-Buffer Input Structure with Boundary Sentinel EOFs

Input Scanning Algorithm with Sentinel Checking:

```
switch (*forward++) {
    case EOF:
        if (forward is at end of first buffer half) {
            reload second buffer half;
            forward = beginning of second half;
        } else if (forward is at end of second buffer half) {
            reload first buffer half;
            forward = beginning of first half;
        } else {
            /* EOF is actual end of source file */
            terminate scanning;
        }
        break;
    /* Process regular characters without secondary boundary checks */
}
```

Topic 7: Thompson's Construction Algorithm (RE $\longrightarrow$ ϵ-NFA)

Thompson's Construction converts any Regular Expression r over alphabet Σ into an equivalent Non-Deterministic Finite Automaton with ϵ-transitions ($N(r)$) in linear time $O(|r|)$.

RE Expression	Automaton Topology & Transitions	Structural Invariant Properties
1. Basic Symbol ($a \in \Sigma$)	State $i \xrightarrow{a}$ State f	Exactly 1 start state, 1 accept state, no outgoing transitions from accept state.
2. Concatenation ($r_1 r_2$)	Accept state of $N(r_1)$ is merged with the start state of $N(r_2)$.	Initial start of $N(r_1)$ becomes overall start; accept of $N(r_2)$ becomes overall accept.
3. Alternation ($r_1 \mid r_2$)	New start state s_0 branches via ϵ to start of $N(r_1)$ and start of $N(r_2)$. Both accept states transition via ϵ to a new single accept state f_0.	Introduces 2 new states and 4 new ϵ-transitions.
4. Kleene Closure (r^*)	New start state s_0 branches via ϵ to start of $N(r)$ and directly to new accept state f_0 (for zero repetitions). Accept of $N(r)$ transitions via ϵ back to start of $N(r)$ and to f_0.	Introduces 2 new states and 4 new ϵ-transitions.

Topic 8: Subset Construction Algorithm (NFA $\longrightarrow$ DFA)

A Deterministic Finite Automaton (DFA) has no ϵ-transitions and has at most one outgoing edge per symbol from any state.

Formal Mathematical Operations in Subset Construction

- $\epsilon\text{-closure}(s)$: The set of all NFA states reachable from state s taking only ϵ-transitions:

$$\epsilon\text{-closure}(s) = \{s\} \cup \{t \mid \exists u \in \epsilon\text{-closure}(s) \text{ such that } u \xrightarrow{\epsilon} t\}$$

- $\epsilon\text{-closure}(T)$: $\bigcup_{s \in T} \epsilon\text{-closure}(s)$ for a set of states T.
- $\text{move}(T, a)$: Set of all NFA states reachable from any state in T on input symbol a:

$$\text{move}(T, a) = \{t \mid \exists s \in T \text{ such that } s \xrightarrow{a} t\}$$

🏛️ Step-by-Step Solved Problem: Convert $(a \mid b)^*abb$ to DFA via Subset Construction

Given NFA State Graph: Start state 0, accept state 10.

Step 1: Compute Initial DFA State $A = \epsilon\text{-closure}(0) = \{0, 1, 2, 4, 7\}$.

Step 2: Iteratively Compute Transitions for All Unmarked DFA States:

DFA State	NFA State Subset	Input 'a'	Input 'b'
A (Start)	$\{0, 1, 2, 4, 7\}$	$\mathbf{B} = \{1, 2, 3, 4, 6, 7, 8\}$	$\mathbf{C} = \{1, 2, 4, 5, 6, 7\}$
B	$\{1, 2, 3, 4, 6, 7, 8\}$	$\mathbf{B} = \{1, 2, 3, 4, 6, 7, 8\}$	$\mathbf{D} = \{1, 2, 4, 5, 6, 7, 9\}$
C	$\{1, 2, 4, 5, 6, 7\}$	$\mathbf{B} = \{1, 2, 3, 4, 6, 7, 8\}$	$\mathbf{C} = \{1, 2, 4, 5, 6, 7\}$
D	$\{1, 2, 4, 5, 6, 7, 9\}$	$\mathbf{B} = \{1, 2, 3, 4, 6, 7, 8\}$	$\mathbf{E}^* = \{1, 2, 4, 5, 6, 7, 10\}$
E* (Accept)	$\{1, 2, 4, 5, 6, 7, 10\}$	$\mathbf{B} = \{1, 2, 3, 4, 6, 7, 8\}$	$\mathbf{C} = \{1, 2, 4, 5, 6, 7\}$

Conclusion: The resulting DFA has 5 states $\{A, B, C, D, E\}$ with E as the unique accepting state.

Topic 9: Direct DFA Construction from Regular Expressions

Direct DFA construction bypasses NFA intermediate generation by decorating the **Augmented Syntax Tree** $(r)\#$ with structural position functions:

Node Type n	$\text{nullable}(n)$	$\text{firstpos}(n)$	$\text{lastpos}(n)$
ϵ-leaf	true	$\emptyset$	$\emptyset$
Leaf position i	false	$\{i\}$	$\{i\}$
$c_1 \mid c_2$ (Or)	$\text{nullable}(c_1) \vee \text{nullable}(c_2)$	$\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$	$\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$
$c_1 \cdot c_2$ (Cat)	$\text{nullable}(c_1) \wedge \text{nullable}(c_2)$	if $\text{nullable}(c_1)$ then $\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$ else $\text{firstpos}(c_1)$	if $\text{nullable}(c_2)$ then $\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$ else $\text{lastpos}(c_2)$
c_1^* (Star)	true	$\text{firstpos}(c_1)$	$\text{lastpos}(c_1)$

Rules for Computing $\text{followpos}(i)$:

1. If n is a Cat-node with left child c_1 and right child c_2, for every position $i \in \text{lastpos}(c_1)$, all positions in $\text{firstpos}(c_2)$ are in $\text{followpos}(i)$.
2. If n is a Star-node with child c_1, for every position $i \in \text{lastpos}(c_1)$, all positions in $\text{firstpos}(c_1)$ are in $\text{followpos}(i)$.

Topic 10: Hopcroft's DFA Minimization Algorithm

Hopcroft's Algorithm computes the unique minimal-state DFA in $O(k \cdot n \log n)$ time by iteratively partitioning states into behavioral equivalence classes:

Initial Partition (P_0): Split all states into Accept (F) and Non-Accept ($S \setminus F$) states:

$$P_0 = \{F,\ S \setminus F\}$$

Refinement: For each group $G \in P$ and symbol $a \in \Sigma$, if $\text{move}(s, a)$ leads states of G into different partition groups, split G into separate sub-groups.
Fixed-Point Convergence: Terminate when $P_{k+1} = P_k$. Collapse each partition group into a single state in the minimal DFA.

Topic 11: Lex & Flex Lexical Analyzer Generator Specifications

Lex Section	Syntax & Contents	Key Lex Variables & Routines
1. Definitions	C headers, `#include`, `%option noyywrap`, regular definitions (`digit [0-9]`).	`yyin` (input file stream pointer), `yyout` (output stream pointer).
2. Translation Rules	Pairs of Regular Expressions and C action blocks: `{digit}+ { yylval = atoi(yytext); return NUM; }`	`yytext` (pointer to matched lexeme string), `yyleng` (length of matched lexeme), `yylval` (semantic attribute value).
3. User Subroutines	Auxiliary C functions, `main()` driver, `yywrap()` handler.	`yylex()` (main DFA scanning function returning integer token codes).

Lex Ambiguity Resolution Rules

1. **Longest Match (Maximal Munch):** The lexer always picks the rule that matches the longest possible sequence of characters (e.g., `<=` is matched as `LE` rather than `<` followed by `=`).
2. **First Match Rule:** If multiple regular expressions match the exact same longest prefix, Lex selects the rule that appears earliest in the `.l` specification file (e.g., `if` keyword listed before generic identifier `[a-z]+`).

🏛️ Top BIT Mesra Exam Questions & Answers (Module I)

Q1. Explain the role of Lexical Analyzer and state 4 reasons why it is separated from the Parser. (8 Marks)

1. **Simplicity of Compiler Architecture:** Separating character-level lexical processing from grammatical phrase parsing simplifies both stages. The parser processes abstract token streams without handling whitespace, tabs, and comments.
2. **Compiler Efficiency:** Specialized high-performance input buffering techniques can be applied to the lexical scanner. Over 70% of compilation CPU cycles are spent scanning characters.
3. **Compiler Portability:** Character-set dependencies (ASCII, UTF-8, Unicode) are isolated strictly inside the lexical phase, making the parser independent of source encoding.
4. **Modular Grammar Design:** Language syntax rules can focus purely on grammar hierarchies rather than token spelling.

Q2. Differentiate between Compiler and Interpreter across 5 core engineering parameters. (6 Marks)

1. **Translation Mechanism:** Compiler translates entire source code into native machine code before execution; Interpreter translates and executes line-by-line at runtime.
2. **Execution Speed:** Compiled machine code executes 10–50× faster than interpreted bytecode.
3. **Memory Usage:** Compiler requires memory for object code storage; Interpreter requires memory for runtime engine.
4. **Error Reporting:** Compiler reports all syntax/semantic errors collectively during compilation; Interpreter stops at the first runtime error encountered.
5. **Examples:** C, C++, Rust (Compiled); Python, Ruby, PHP (Interpreted).

Q3. Trace the Direct DFA construction for regular expression $(a \mid b)^* abb\#$. (10 Marks)

1. **Augmented Syntax Tree:** Leaf positions: $1:a, 2:b, 3:a, 4:b, 5:b, 6:\#$.
2. **Firstpos / Lastpos:** $\text{firstpos}(\text{root}) = \{1, 2, 3\}$.
3. **Followpos Table:** - $\text{followpos}(1) = \{1, 2, 3\}$ - $\text{followpos}(2) = \{1, 2, 3\}$ - $\text{followpos}(3) = \{4\}$ - $\text{followpos}(4) = \{5\}$ - $\text{followpos}(5) = \{6\}$ 4. **DFA States:** State $A = \{1, 2, 3\}$; State $B = \{1, 2, 3, 4\}$; State $C = \{1, 2, 3, 5\}$; State $D^* = \{1, 2, 3, 6\}$ (Accept).

Module II: Syntax Analysis & Parsing Algorithms — Complete Syllabus Topics

1. Role of Parser & Context-Free Grammars (CFG) Mathematical Formulations
2. Parse Trees, Derivations (Leftmost & Rightmost) & Ambiguity Proofs
3. Left Recursion Elimination (Immediate & Indirect Grammatical Algorithms)
4. Left Factoring Algorithm for Common Prefixes
5. FIRST and FOLLOW Sets Formal Inductive Formulations & Traces
6. LL(1) Top-Down Predictive Parsing Table Construction & Stack Simulations
7. Bottom-Up Shift-Reduce Parsing Principles & Handle Pruning Theorems
8. LR(0) Canonical Collection of Items & SLR(1) Parsing Table Formulations
9. Canonical LR (CLR(1)) Items with Lookaheads & LALR(1) State Merging
10. Parsing Conflicts (Shift/Reduce, Reduce/Reduce) & Ambiguity Resolutions
11. Syntax Error Detection & Recovery Strategies (Panic Mode & Phrase Level)
12. Comprehensive Solved BIT Mesra & GATE Question Bank (8 Solved Problems)

Topic 1 & 2: Context-Free Grammars, Derivations & Ambiguity

A **Context-Free Grammar (CFG)** is formally defined as a 4-tuple $G = (V, T, P, S)$:

V: Finite set of Non-Terminal characters (syntactic variables).
T: Finite set of Terminal characters (token types returned by lexical analyzer).
P: Finite set of Production rules of the form $A \to \alpha$, where $A \in V$ and $\alpha \in (V \cup T)^*$.
$S \in V$: The designated Start Symbol.

1. Derivations & Parse Trees:

Leftmost Derivation (LMD): At each derivation step, the leftmost non-terminal is replaced by the body of one of its productions ($\alpha \underset{lm}{\Longrightarrow} \beta$).
Rightmost Derivation (RMD / Canonical Derivation): At each step, the rightmost non-terminal is replaced ($\alpha \underset{rm}{\Longrightarrow} \beta$). Bottom-up parsers reconstruct the reverse of a rightmost derivation (RMD^R).
Parse Tree: A graphical representation of a derivation where the root is start symbol S, interior nodes are non-terminals, and leaves are terminals or ϵ, read left-to-right to yield the input sentence.

> **Formal Definition of Grammar Ambiguity**
> A grammar G is **ambiguous** if there exists at least one string $w \in L(G)$ that has **two or more distinct parse trees** (or equivalently, two distinct leftmost derivations). Ambiguity makes deterministic parsing impossible unless resolved by disambiguating rules.

🏛️ Classic Problem: Disambiguating the Dangling-Else Grammar

Ambiguous Grammar:

$$\text{stmt} \rightarrow \textbf{if expr then stmt} \mid \textbf{if expr then stmt else stmt} \mid \textbf{other}$$

For input `if E1 then if E2 then S1 else S2`, the `else` clause can be attached to the first or second `if`.

Unambiguous Disambiguated Grammar:

```
stmt          → matched_stmt | open_stmt
matched_stmt  → if expr then matched_stmt else matched_stmt | other
open_stmt     → if expr then stmt
              | if expr then matched_stmt else open_stmt
```

Enforces that every `else` matches the closest preceding unmatched `then`.

Topic 3 & 4: Grammar Transformations (Left Recursion & Left Factoring)

1. Immediate Left Recursion Elimination:

A production of the form $A \rightarrow A\alpha \mid \beta$ causes top-down predictive parsers to loop infinitely. It is eliminated by introducing a new non-terminal A':

$$A \rightarrow \beta A', \qquad A' \rightarrow \alpha A' \mid \epsilon$$

2. General Left Recursion Elimination Algorithm (Handling Indirect Recursion):

Order non-terminals in an arbitrary sequence: $A_1, A_2, \ldots, A_n$.
For $i = 1$ to n:
For $j = 1$ to $i - 1$: Replace each production of the form $A_i \rightarrow A_j\gamma$ by $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \cdots \mid \delta_k\gamma$, where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \cdots \mid \delta_k$ are the current A_j rules.
Eliminate immediate left recursion among the A_i productions.

3. Left Factoring Algorithm (Removing Common Prefixes):

When two productions for A share a common prefix ($A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$), the parser cannot determine which rule to apply with 1 lookahead token. Left factoring delays the decision:

$$A \rightarrow \alpha A', \qquad A' \rightarrow \beta_1 \mid \beta_2$$

Topic 5: Formal Formulations of FIRST and FOLLOW Sets

Mathematical Definitions

- $\text{FIRST}(\alpha)$: Set of all terminals that appear as the first symbol in strings derived from α. If $\alpha \xrightarrow{*} \epsilon$, then $\epsilon \in \text{FIRST}(\alpha)$.
- $\text{FOLLOW}(A)$: Set of all terminals a that can appear immediately to the right of non-terminal A in some sentential form ($S \xrightarrow{*} \alpha A a \beta$). If A is the rightmost symbol in a derivation, the input end-marker \$ is in $\text{FOLLOW}(A)$.

Formal Rules for Computing FIRST:

If X is a terminal, $\text{FIRST}(X) = \{X\}$.
If $X \rightarrow \epsilon$ is a production, add ϵ to $\text{FIRST}(X)$.
If $X \rightarrow Y_1 Y_2 \ldots Y_k$ is a production:
Add all non-ϵ symbols of $\text{FIRST}(Y_1)$ to $\text{FIRST}(X)$.

If $\epsilon \in \text{FIRST}(Y_1)$, add non-$\epsilon$ symbols of $\text{FIRST}(Y_2)$, continuing until some Y_i does not contain ϵ.
If all $Y_1, \ldots, Y_k$ contain ϵ, add ϵ to $\text{FIRST}(X)$.

Formal Rules for Computing FOLLOW:

Place \$ in $\text{FOLLOW}(S)$, where S is the grammar start symbol.
If there is a production $A \to \alpha B \beta$, everything in $\text{FIRST}(\beta)$ except ϵ is in $\text{FOLLOW}(B)$.
If there is a production $A \to \alpha B$, or $A \to \alpha B \beta$ where $\text{FIRST}(\beta)$ contains ϵ, everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$.

🏛 Step-by-Step Solved Problem: Arithmetic Grammar FIRST & FOLLOW

Transformed Arithmetic Grammar (G):

- $E \to TE'$
- $E' \to +TE' \mid \epsilon$
- $T \to FT'$
- $T' \to *FT' \mid \epsilon$
- $F \to (E) \mid \mathbf{id}$

Non-Terminal	FIRST Set	FOLLOW Set
E	$\{(, \mathbf{id}\}$	$\{), \$\}$
E'	$\{+, \epsilon\}$	$\{), \$\}$
T	$\{(, \mathbf{id}\}$	$\{+,), \$\}$
T'	$\{*, \epsilon\}$	$\{+,), \$\}$
F	$\{(, \mathbf{id}\}$	$\{*, +,), \$\}$

Topic 6: Top-Down LL(1) Predictive Parsing

An **LL(1) Parser** scans input from **L**eft-to-right, produces a **L**eftmost derivation, using **1** token of lookahead without backtracking.

Formal Condition for an LL(1) Grammar

A context-free grammar G is $LL(1)$ if and only if for every pair of productions $A \to \alpha \mid \beta$:

1. $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$.
2. If $\epsilon \in \text{FIRST}(\alpha)$, then $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$.

Complete LL(1) Parsing Table $M[A, a]$:

Non-Terminal	$\mathbf{id}$	$+$	$*$	$($	$)$	$\$$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to \mathbf{id}$			$F \to (E)$		

Since no cell contains multiple production rules, the grammar is strictly $LL(1)$.

Topics 7 – 9: Bottom-Up LR Parsing Hierarchy

LR Parsers (Left-to-right scan, Rightmost derivation in reverse) are the most general deterministic shift-reduce parsing engines.

Figure 2.1: Relative Grammatical Expressive Power of LR Parser Classes

Parser Class	Item Representation	Reduce Action Decision Logic	State Complexity & Practical Use
1. LR(0)	$[A \to \alpha \cdot \beta]$	Reduce $A \to \alpha$ across all columns in row unconditionally.	Fewest states; very weak; fails on almost all programming grammars.
2. SLR(1)	$[A \to \alpha \cdot \beta]$	Reduce $A \to \alpha$ only on input terminal $a \in \text{FOLLOW}(A)$.	Same states as LR(0); resolves many simple shift/reduce conflicts.
3. CLR(1)	$[A \to \alpha \cdot \beta,\ a]$ (with explicit lookahead a)	Reduce $A \to \alpha$ only on exact lookahead terminal a.	Most powerful deterministic class; massive state explosion (thousands of states).
4. LALR(1)	Merged LR(1) items having identical LR(0) core items.	Reduce on merged lookahead terminal sets.	Same number of states as SLR(1); nearly CLR(1) parsing power; standard in Yacc/Bison.

Topic 10 & 11: Parsing Conflicts & Error Recovery

1. Bottom-Up Parsing Conflicts:

Shift/Reduce (S/R) Conflict: The parser state cannot decide whether to shift the next input symbol onto the stack or reduce the current top-of-stack handle by a production rule.
Reduce/Reduce (R/R) Conflict: The parser state recognizes two or more distinct completed handles simultaneously and cannot decide which production to reduce by (indicates severe grammatical ambiguity).

2. Error Recovery Modes:

Panic-Mode Recovery: The parser discards incoming tokens until a designated synchronizing token (e.g., `;`, `}`) is encountered, then pops stack symbols until parsing can safely resume. Simple and guaranteed to avoid infinite loops.
Phrase-Level Recovery: Replaces an erroneous prefix with a valid correction (e.g., replacing comma with semicolon).
Error Productions: Language designers augment the grammar with common anticipated developer errors to emit detailed diagnostic messages.

🏛️ Top BIT Mesra Exam Questions & Answers (Module II)

Q1. Prove why an ambiguous grammar can never be LL(1) or LR(1). (8 Marks)

Proof: An ambiguous grammar generates at least two distinct parse trees for some valid string w.
1. In an **LL(1) parser**, this ambiguity manifests as multiple production entries in the same parsing table cell $M[A, a]$, violating the single-entry requirement of deterministic LL(1) tables.
2. In an **LR(1) parser**, the two distinct derivations create either a **Shift/Reduce conflict** (where the parser cannot decide whether to shift or reduce) or a **Reduce/Reduce conflict** (where the parser cannot decide which of two distinct productions to reduce by). Thus, ambiguous grammars can never be parsed deterministically by LL(1) or LR(1).

Q2. Explain the difference between SLR(1) and LALR(1) parsers. Can merging states introduce Shift/Reduce conflicts in LALR(1)? (10 Marks)

- **SLR(1):** Uses LR(0) items and makes reduction decisions based on the global $\text{FOLLOW}(A)$ set, which often includes symbols that cannot legally appear in that specific context.
- **LALR(1):** Merges CLR(1) states having identical LR(0) cores while unioning their lookahead sets. Reduction is guided by exact propagated lookaheads.
- **Theorem on Conflicts:** Merging LR(1) states with identical cores **CANNOT introduce new Shift/Reduce conflicts** (because shift decisions depend solely on the core item's symbol after the dot). However, merging **CAN introduce new Reduce/Reduce conflicts** if two distinct productions have overlapping lookaheads after merging.

Module III: Semantic Analysis & Type Checking — Complete Syllabus Topics

1. Role of Semantic Analysis & Static vs. Dynamic Checking
2. Syntax-Directed Definitions (SDD) & Semantic Rule Formulations
3. Synthesized Attributes vs. Inherited Attributes Formal Properties
4. S-Attributed Definitions & Bottom-Up LR Parsing Implementations
5. L-Attributed Definitions & Depth-First Left-to-Right Traversal Orders
6. Dependency Graphs & Topological Sorting for Attribute Evaluation
7. Syntax-Directed Translation Schemes (SDTS) & Action Placement Rules
8. Type Systems, Type Expressions & Static Type Checker Architecture
9. Type Equivalence (Structural vs. Name Equivalence Algorithms)
10. Type Conversions (Implicit Coercions vs. Explicit Casting Mechanics)
11. Symbol Table Organizations (Block Scoping with Chained Hash Tables)
12. Comprehensive Solved BIT Mesra & GATE Question Bank (8 Solved Problems)

Topic 1 & 2: Syntax-Directed Definitions (SDD) & Attribute Types

A **Syntax-Directed Definition (SDD)** is a context-free grammar augmented with attributes and semantic rules. Attributes are associated with grammar symbols, and semantic rules are associated with production rules.

Attribute Class	Formal Mathematical Formulation & Value Flow	Parsing & Evaluation Invariants
1. Synthesized Attribute	Computed strictly from the attribute values of the node's children in the parse tree (or lexical values): $A.s = f(Y_1.a_1, Y_2.a_2, \ldots, Y_k.a_k) \quad \text{for production } A \to Y_1 Y_2 \ldots Y_k$	Flows strictly **Bottom-Up**; naturally evaluated during shift-reduce / LR parsing using an attribute stack.
2. Inherited Attribute	Computed from the attribute values of the node's parent and/or left siblings: $Y_j.i = f(A.a, Y_1.a_1, \ldots, Y_{j-1}.a_{j-1}) \quad \text{for symbol } Y_j \text{ in } A \to Y_1 \ldots Y_k$	Flows **Top-Down** and **Left-to-Right**; passes context (e.g., base data types) down into declarations.

Topic 3 & 4: S-Attributed vs. L-Attributed SDD

Structural Classifications

- **S-Attributed SDD:** An SDD that uses **only synthesized attributes**. Can be evaluated during bottom-up parsing (e.g., LR parser) by maintaining an attribute value stack in parallel with the parser state stack.
- **L-Attributed SDD:** An SDD where every attribute is either synthesized, or inherited with the restriction that for production $A \to X_1 X_2 \ldots X_n$, an inherited attribute of X_j depends only on:

1. Attributes of the parent A.
2. Attributes of symbols to the left of X_j ($X_1, X_2, \ldots, X_{j-1}$).

Every S-Attributed SDD is strictly an L-Attributed SDD ($S\text{-Attributed} \subset L\text{-Attributed}$).

🏛️ Step-by-Step Solved Problem: Desktop Calculator S-Attributed SDD

Grammar Production	Semantic Evaluation Rule
$L \to E\ \mathbf{n}$	$\text{print}(E.\text{val})$
$E \to E_1 + T$	$E.\text{val} = E_1.\text{val} + T.\text{val}$
$E \to T$	$E.\text{val} = T.\text{val}$
$T \to T_1 * F$	$T.\text{val} = T_1.\text{val} * F.\text{val}$
$T \to F$	$T.\text{val} = F.\text{val}$
$F \to (E)$	$F.\text{val} = E.\text{val}$
$F \to \mathbf{digit}$	$F.\text{val} = \mathbf{digit}.\text{lexval}$

Evaluates expression `(3 + 4) * 5\n` bottom-up in a single LR parsing pass to yield `35`.

Topic 6: Dependency Graphs & Topological Evaluation Orders

A **Dependency Graph** represents the data-flow constraints between attribute instances in a parse tree:

For each node attribute $X.a$, there is a corresponding graph vertex.
A directed edge $X.a \to Y.b$ indicates that the semantic rule for $Y.b$ requires the value of $X.a$.
If the dependency graph contains **no directed cycles**, a valid evaluation order is given by any **Topological Sort** of the graph.

Topic 8 & 9: Type Systems & Type Equivalence

1. Structural vs. Name Equivalence:

Name Equivalence: Two types are equivalent if and only if they share the exact same named type identifier.

```
typedef struct { int x, y; } PointA;
typedef struct { int x, y; } PointB;
// Under Name Equivalence: PointA ≠ PointB (Strict)
```

Structural Equivalence: Two types are equivalent if they have identical internal constituent structures.

```
// Under Structural Equivalence: PointA == PointB (Both contain two integer fields)
```

2. Type Conversions & Coercion Rules:

Widening Conversions (Implicit): Preserve numeric precision (e.g., `char` → `int` → `float` → `double`). Automatically inserted by compiler.
Narrowing Conversions (Explicit): May lose precision or overflow (e.g., `double` → `int`). Require explicit type cast syntax.

Topic 11: Symbol Table Organizations (Block Scoping with Hash Tables)

Figure 3.1: Hierarchical Chained Hash Tables for Nested Lexical Block Scoping

🏛️ Top BIT Mesra Exam Questions & Answers (Module III)

Q1. Differentiate between S-Attributed and L-Attributed SDD with examples and evaluation strategies. (8 Marks)

1. **S-Attributed SDD:** Uses only synthesized attributes. Evaluated strictly bottom-up during LR parsing using an attribute value stack without needing an explicit syntax tree.
2. **L-Attributed SDD:** Allows synthesized and inherited attributes with the restriction that inherited attributes of X_j in $A \to X_1 X_2 \dots X_n$ depend only on attributes of parent A and left siblings $X_1, \dots, X_{j-1}$. Evaluated in a single depth-first, left-to-right tree traversal.

Q2. Write an SDD to translate type declarations like `int a, b, c;` into symbol table entries. (8 Marks)

Production	Semantic Rule
$D \to T\ L$	$L.\text{in} = T.\text{type}$
$T \to \textbf{int}$	$T.\text{type} = \text{integer}$
$T \to \textbf{float}$	$T.\text{type} = \text{real}$
$L \to L_1,\ \textbf{id}$	$L_1.\text{in} = L.\text{in};\ \text{addType}(\textbf{id}.\text{entry}, L.\text{in})$
$L \to \textbf{id}$	$\text{addType}(\textbf{id}.\text{entry}, L.\text{in})$

Uses inherited attribute $L.\text{in}$ to propagate base type downward to all identifier instances.

Module IV: Intermediate Code Generation & Runtime Environments — Complete Topics

1. Intermediate Representations (Syntax Trees, DAG, Postfix, TAC)
2. Three-Address Code (TAC) Representations: Quadruples, Triples, Indirect Triples
3. Translation of Arithmetic Expressions & Type Coercion TAC
4. Multi-Dimensional Array Addressing (Row-Major vs. Column-Major Proofs)
5. Boolean Expressions Translation & Flow-of-Control Short-Circuiting
6. Backpatching Techniques (`makelist`, `merge`, `backpatch` Algorithms)
7. Procedure Translation (`param`, `call`, `return` Instruction Sequences)
8. Runtime Storage Organization (Text, Static, Stack & Heap Segments)
9. Activation Records (Stack Frame Anatomy & Dynamic Links)
10. Access Links & Displays for Lexically Nested Scopes
11. Parameter Passing Mechanisms (Value, Reference, Copy-Restore, Name)
12. Comprehensive Solved BIT Mesra & GATE Question Bank (8 Solved Problems)

Topic 1 & 2: Three-Address Code (TAC) Representations

Three-Address Code (TAC) is a linear intermediate representation where each instruction contains at most one operator on the right-hand side:

$$x = y \ \mathbf{op} \ z$$

Representation	Internal Data Structure & Fields	Key Advantages & Tradeoffs
1. Quadruples	Four fields: `(op, arg1, arg2, result)`. Explicit temporary names (e.g., `t1`, `t2`) are stored in `result`.	Easy to reorder during code optimization; requires extra memory for temporary names.
2. Triples	Three fields: `(op, arg1, arg2)`. Results of operations are referred to by their instruction index position (e.g., `(0)`, `(1)`).	Space efficient; moving or inserting instructions requires updating all index references.
3. Indirect Triples	Uses an array of pointers to separate triple structures.	Optimizers can reorder instructions by simply swapping pointers in the array without touching the underlying triples.

🏛️ Step-by-Step Solved Problem: Quadruples, Triples & Indirect Triples for `a = b * - c + b * - c`

Step 1: Generate Three-Address Code:

```
(0) t1 = uminus c
(1) t2 = b * t1
(2) t3 = uminus c
(3) t4 = b * t3
(4) t5 = t2 + t4
(5) a = t5
```

Step 2: Quadruple Representation:

#	op	arg1	arg2	result
0	uminus	c	-	t1
1	*	b	t1	t2
2	uminus	c	-	t3
3	*	b	t3	t4
4	+	t2	t4	t5
5	=	t5	-	a

Step 3: Triple Representation:

#	op	arg1	arg2
(0)	uminus	c	-
(1)	*	b	(0)
(2)	uminus	c	-
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	assign	a	(4)

Topic 4: Multi-Dimensional Array Addressing Formulations

1. 1D Array Address Formula:

$$\text{Address}(A[i]) = \text{base} + (i - \text{low}) \times w$$

Where base is the start memory address, low is lower index bound, and w is byte width per element.

2. 2D Array Address Formula (Row-Major Order — C / C++ / Java):

$$\text{Address}(A[i_1, i_2]) = \text{base} + \Big((i_1 - \text{low}_1) \times n_2 + (i_2 - \text{low}_2)\Big) \times w$$

Where $n_2 = \text{high}_2 - \text{low}_2 + 1$ is the number of columns.

3. 2D Array Address Formula (Column-Major Order — FORTRAN / MATLAB):

$$\text{Address}(A[i_1, i_2]) = \text{base} + \Big((i_2 - \text{low}_2) \times n_1 + (i_1 - \text{low}_1)\Big) \times w$$

Where $n_1 = \text{high}_1 - \text{low}_1 + 1$ is the number of rows.

Topic 5 & 6: Backpatching in Boolean Expressions & Control Flow

Backpatching is a single-pass technique for generating intermediate code for control flow and boolean expressions without leaving unresolved forward jump target addresses:

`makelist(i)`: Creates a new list containing only jump instruction index i.
`merge(p1, p2)`: Concatenates two jump target lists p_1 and p_2.
`backpatch(p, i)`: Inserts target instruction label i as the jump destination into all instructions indexed in list p.

Topics 8 – 10: Runtime Storage Organization & Activation Records

During program execution, the operating system allocates a contiguous block of virtual memory partitioned into 4 logical segments:

Code Segment (Text): Read-only region holding compiled target machine instructions.
Static / Global Segment: Holds global variables, static constants, and compiler metadata fixed at compile time.
Call Stack: Grows downward dynamically; stores **Activation Records (Stack Frames)** for function invocations.
Heap: Grows upward dynamically; handles explicit dynamic memory allocation (`malloc`, `free`, `new`).

Activation Record Field	Functional Purpose in Procedure Call / Return
Actual Parameters	Arguments passed by the caller to the callee.
Returned Values	Space for return data passed back to caller.
Control Link (Dynamic Link)	Points to caller's activation record; used to restore caller's stack frame on return.
Access Link (Static Link)	Points to activation record of the statically enclosing lexical scope; used to access non-local variables.
Saved Machine Status	Program counter (PC), CPU registers, and status flags saved before function call.
Local Data	Local automatic variables allocated within the function.
Temporaries	Temporary values generated during expression evaluation.

🏛️ Top BIT Mesra Exam Questions & Answers (Module IV)

Q1. Compare Quadruples, Triples, and Indirect Triples with examples. (8 Marks)

1. **Quadruples:** Explicit 4-field structures `(op, arg1, arg2, res)` where temporary names (`t1`) are explicitly stated. Highly flexible for compiler optimization reordering, but uses more memory.
2. **Triples:** 3-field structures `(op, arg1, arg2)` where intermediate results are referenced by instruction array indices `(0)`. Memory compact, but relocating code requires patching all dependent instruction references.
3. **Indirect Triples:** Stores pointers to triple structures in an array. Allows optimizers to reorder and eliminate code simply by modifying pointer positions without touching underlying triple records.

Q2. Explain Call-by-Value, Call-by-Reference, and Call-by-Name parameter passing. (6 Marks)

- **Call-by-Value:** Actual parameter expression is evaluated and a local copy of the value is passed to the callee. Changes inside the function do not affect the caller's variable.
- **Call-by-Reference:** Memory address (pointer) of the actual parameter is passed. Any modification inside the function directly modifies caller's variable.
- **Call-by-Name (Algol 60):** The parameter is not evaluated upfront; instead, the parameter text is literally substituted into the function body (evaluated lazily via thunks every time it is referenced).

Module V: Code Optimization & Target Code Generation — Complete Syllabus Topics

1. Principal Sources of Optimization (Machine-Independent Transformations)
2. Basic Blocks Partitioning Algorithm & Control Flow Graphs (CFG) Construction
3. Directed Acyclic Graphs (DAG) Representation for Basic Blocks & Transformations
4. Local vs. Global Optimization Strategies & Common Subexpression Elimination
5. Loop Optimizations (Loop Invariant Code Motion, Strength Reduction, Unrolling)
6. Dominator Trees, Natural Loops & Reducible Flow Graph Properties
7. Data-Flow Analysis Frameworks (Reaching Definitions & Available Expressions)
8. Live Variable Analysis Equations & Dead Code Elimination Passes
9. Target Code Generation Issues & Register Allocation by Graph Coloring
10. Simple Code Generator Algorithm using Register & Address Descriptors
11. Peephole Optimization Techniques (Redundant Load/Store & Algebraic Identities)
12. Comprehensive Solved BIT Mesra & GATE Question Bank (8 Solved Problems)

Topic 1 & 4: Principal Sources of Code Optimization

Code optimization transforms intermediate or target code to run faster, consume less memory, or use less power, while strictly preserving program semantics.

Optimization Technique	Mechanisms & Transformations	Before → After Example
1. Constant Folding	Evaluating operations on compile-time constants upfront.	`x = 3 + 4 * 2` → `x = 11`
2. Constant Propagation	Replacing variables holding known constant values with the constant.	`pi = 3.14; r = 2; area = pi * r * r` → `area = 3.14 * 2 * 2`
3. Common Subexpression Elimination	Reusing previously computed values instead of re-evaluating identical expressions.	`t1 = a + b; ... ; t2 = a + b` → `t2 = t1`
4. Dead Code Elimination	Removing instructions whose results are never read by subsequent code or unreachable blocks.	`if (0) { do_something(); }` → `/* Removed */`
5. Strength Reduction	Replacing expensive operations (e.g., multiplication) with cheaper equivalents (addition, bit-shift).	`x = y * 8` → `x = y << 3`
6. Loop Invariant Code Motion	Hoisting computations that evaluate to the same value in every loop iteration outside the loop header.	`while (i < n) { x = a + b; i++; }` → `x = a + b; while (i < n) { i++; }`

Topic 2: Basic Blocks & Control Flow Graphs (CFG)

Algorithm: Partitioning Three-Address Code into Basic Blocks

Step 1: Identify the Leaders (first instructions of basic blocks):

1. The first Three-Address instruction of the intermediate code is a Leader.
2. Any instruction that is the target of a conditional or unconditional jump (`goto L`) is a Leader.
3. Any instruction that immediately follows a conditional or unconditional jump is a Leader.

Step 2: Construct Basic Blocks:

For each Leader, its Basic Block consists of the Leader and all subsequent instructions up to, but not including, the next Leader or the end of the program.

🏛️ Step-by-Step Solved Problem: Basic Block Partitioning on Dot Product Code

Three-Address Code:

```
(1)  prod = 0          ⟵ Leader (Rule 1: First instruction)
(2)  i = 1
(3)  t1 = 4 * i        ⟵ Leader (Rule 2: Target of goto (3) in line 10)
(4)  t2 = a[t1]
(5)  t3 = 4 * i
(6)  t4 = b[t3]
(7)  t5 = t2 * t4
(8)  prod = prod + t5
(9)  i = i + 1
(10) if i ⩽ 20 goto (3)
(11) print(prod)       ⟵ Leader (Rule 3: Follows conditional jump at line 10)
```

Resulting Basic Blocks:

- **Block B_1:** Instructions (1) to (2).
- **Block B_2 (Loop Body):** Instructions (3) to (10).
- **Block B_3:** Instruction (11).

Topic 3: Directed Acyclic Graphs (DAG) for Basic Blocks

A **DAG (Directed Acyclic Graph)** is an efficient representation for basic blocks that makes local common subexpression elimination, dead code elimination, and array dependency tracking straightforward:

Leaves correspond to initial values of variables or constants.
Interior nodes correspond to operators.
Nodes have labels attached showing current identifier names having that computed value.

Topic 7 & 8: Data-Flow Analysis Frameworks

Framework	Data-Flow Transfer Equations	Direction & Confluence Operator
1. Reaching Definitions	$$\text{OUT}[B] = \text{GEN}[B] \cup (\text{IN}[B] \setminus \text{KILL}[B])$$ $$\text{IN}[B] = \bigcup_{P \in \text{Pred}(B)} \text{OUT}[P]$$	Forward Data-Flow; Union ($\cup$) confluence.
2. Available Expressions	$$\text{OUT}[B] = e\_\text{GEN}[B] \cup (\text{IN}[B] \setminus e\_\text{KILL}[B])$$ $$\text{IN}[B] = \bigcap_{P \in \text{Pred}(B)} \text{OUT}[P]$$	Forward Data-Flow; Intersection ($\cap$) confluence.
3. Live Variable Analysis	$$\text{IN}[B] = \text{USE}[B] \cup (\text{OUT}[B] \setminus \text{DEF}[B])$$ $$\text{OUT}[B] = \bigcup_{S \in \text{Succ}(B)} \text{IN}[S]$$	Backward Data-Flow; Union ($\cup$) confluence.

Topic 9 & 11: Target Code Generation & Peephole Optimization

1. Register Allocation by Graph Coloring (Chaitin's Algorithm):

Construct the **Interference Graph** where nodes represent variables/temporaries and edges connect variables that are simultaneously live at the same program point.
Find a K-coloring of the graph, where K is the number of available physical hardware CPU registers. If successful, no two interfering variables share the same register.
If the graph cannot be K-colored, select candidate variables to **spill** into RAM stack slots.

2. Peephole Optimization:

A local optimization method that examines a short slide window of target assembly instructions ("peephole") and applies immediate algebraic and redundant instruction eliminations:

Eliminating Redundant Loads and Stores:

```
MOV R0, a
MOV a, R0     ; Redundant - Eliminated!
```

Eliminating Unreachable Code: Instructions immediately following an unconditional jump without a label are deleted.
Algebraic Simplifications: Replacing $x + 0 \rightarrow x, x * 1 \rightarrow x, x * 0 \rightarrow 0$.

🏛️ Top BIT Mesra Exam Questions & Answers (Module V)

Q1. State the algorithm to identify Basic Blocks and construct a Control Flow Graph. (8 Marks)

1. **Identify Leaders:** (i) First instruction of program, (ii) Target of any jump (`goto`), (iii) Instruction immediately following any jump.
2. **Construct Blocks:** For each leader, its basic block consists of all instructions up to the next leader or program end.
3. **Build Control Flow Graph (CFG):** Nodes are the basic blocks. Directed edges $B_i \rightarrow B_j$ exist if control can transfer from B_i to B_j (via conditional/unconditional jump or sequential fall-through).

Q2. Explain Loop Invariant Code Motion and Strength Reduction with code examples. (8 Marks)

- **Loop Invariant Code Motion:** Moves computations whose operand values do not change across loop iterations outside the loop header.
Example: `for (int i=0; i - **Strength Reduction:** Replaces computationally expensive operations (e.g., multiplication inside loops driven by induction variables) with cheaper operations (addition).
Example: `for (int i=0; i<100; i++) { t = i * 4; }` $\rightarrow$ `t = 0; for (int i=0; i<100; i++) { ... t += 4; }`